Parker Lovill

Student ID#: 00127715

09/24/2026

CS514

Assignment03: Fun with Parsing

Analysis Questions:

1. Why does application have higher precedence than +?

Application has precedence over plus, which implies that in a line such as f x + g y, you apply f to x, and add the resulting value to g y. Evaluate the function calls first, then perform the addition. This is how humans normally interpret lines with mixed math and function calls.

2. Why is f x y left associative?

f x y means you apply f to x and then use this result on y. It is left-associative, so you first get a new function using f on x and then apply this new function to y. This is how curried calls happen in this language.

3. Why is a function AST node not enough to represent a function value?

The syntax of a Fun node consists of just the code itself. The Fun node has the parameter name and the body tree. No information about the meaning of the free variables in the body tree is contained in the Fun node. This information comes from the environment in which the function is defined.

4. What information must a closure contain?

Three ingredients are required for this. The first thing needed is the name of the parameter, which allows the interpreter to know what to bind to when calling the function. The second one is the body of the function, which remains as code until it is called.

5. Why does function application use the defining environment?

The value of a free identifier is supposed to be whatever it was when the procedure was defined, and not the value it happens to have at the time that the procedure gets called. This is lexical scope.

6. What output would reveal an accidental dynamic-scope implementation?

In other words: let x = 10. Now create function f defined by y -> x + y. Next let x = 100 and apply function f to argument 5. At the time of definition of f, value of x was 10, thus applying f to 5 we will get (10 + 5) and output 15.

7. Why does currying create closures naturally?

fun x y -> x + y is a function which returns another function. Two Fun nodes will be created by the parser, where the first one binds x, while the second one binds y and adds them up.

8. How does the closure interpreter avoid capture-avoiding substitution?

Beta-reduction using copy in the old lambda calculus is achieved by iterating through the body and replacing all the free occurrences of x with the argument term. In the case where there is a free occurrence of a name in the argument that is bound by the body, then that particular name is captured. The argument now makes references to the newly bound occurrence rather than the one being replaced.

9. What additional machinery would be needed for recursive named functions?

The name fact should already be present in the environment in which the closure stores the fact. However, when the closure is called, it will refer to fact in its body. But before storing the closure in the environment, it has to exist first.

10. How could the named AST be translated to de Bruijn indices instead?

De Bruijn indices represent the number of function binders between the occurrence and definition of that name. Rather than referring to the name x, it refers to an integer. The

expression fun x -> fun y -> x is therefore represented by lambda lambda 1. The inner binder, y, is 0 as it is closest, while x, which is next in order, is 1.

Output Screenshot:

